IPFS 노드 초기화 과정 상세 분석

개요

IPFS 노드 초기화는 `ipfs init` 명령어를 통해 수행되며, 새로운 IPFS 노드를 위한 모든 필수 구성 요소를 생성하고 설정하는 과정입니다. 이 과정에는 암호화 키 생성, 설정 파일 작성, 저장소 초기화, 기본 문서 추가 등이 포함됩니다.

초기화 명령어 구조

기본 명령어 형식

```
ipfs  init [options] [default-config]
```

주요 옵션

- `--algorithm, -a` : 키 생성에 사용할 암호화 알고리즘 (기본값: ed25519)
- `--bits, -b` : RSA 키의 비트 수 (RSA 사용 시)
- `--empty-repo, -e` : 도움말 파일을 추가하지 않음 (기본값: false)
- `--profile, -p` : 적용할 설정 프로파일 (쉼표로 구분하여 여러 개 지정 가능)

초기화 과정 단계별 분석

1단계: 사전 검증

저장소 존재 여부 확인

```
func doInit(out io.Writer, repoRoot string, empty bool, confProfiles string,
if _, err := fmt.Fprintf(out, "initializing IPFS node at %s\n", repoRoot); err
return err
}

if err := checkWritable(repoRoot); err != nil {
return err
}

if fsrepo.IsInitialized(repoRoot) {
return errRepoExists
}
// ...
}
```

쓰기 권한 검증

```
func checkWritable(dir string) error {
_, err := os.Stat(dir)
if err == nil {
// 디렉토리가 존재하는 경우, 쓰기 권한 테스트
testfile := filepath.Join(dir, "test")
fi, err := os.Create(testfile)
```

```
if err != nil {
if os.IsPermission(err) {
return fmt.Errorf("%s is not writeable by the current user", dir)
}
return fmt.Errorf("unexpected error while checking writeablility of repo root:
}
fi.Close()
return os.Remove(testfile)
}


if os.IsNotExist(err) {
// 디렉토리가 없는 경우, 생성 가능 여부 확인
return os.Mkdir(dir, 0o775)
}
// ...
}
```

2단계: 신원(Identity) 생성

암호화 키 쌍 생성

```
func CreateIdentity(out io.Writer, opts []options.KeyGenerateOption) (Identity,
ident := Identity{}
```

```
settings, err := options.KeyGenerateOptions(opts...)
if err != nil {
return ident, err
}

var sk crypto.PrivKey
var pk crypto.PubKey

switch settings.Algorithm {
case "rsa":
if settings.Size == -1 {
settings.Size = options.DefaultRSALen
}
fmt.Fprintf(out, "generating %d-bit RSA keypair...", settings.Size)
priv, pub, err := crypto.GenerateKeyPair(crypto.RSA, settings.Size)
if err != nil {
return ident, err
}
sk = priv
pk = pub
case "ed25519":
if settings.Size != -1 {
return ident, fmt.Errorf("number of key bits does not apply when using ed25519
```

```
}
fmt.Fprintf(out, "generating ED25519 keypair...")
priv, pub, err := crypto.GenerateEd25519Key(rand.Reader)
if err != nil {
return ident, err
}
sk = priv
pk = pub
}
fmt.Fprintf(out, "done\n")
// ...
}
```

피어 ID 생성

```
// 개인키를 Base64로 인코딩하여 저장
skbytes, err := crypto.MarshalPrivateKey(sk)
if err != nil {
return ident, err
}
ident.PrivKey = base64.StdEncoding.EncodeToString(skbytes)

// 공개키로부터 피어 ID 생성
```

```
id, err := peer.IDFromPublicKey(pk)

if err != nil {

return ident, err

}

ident.PeerID = id.String()

fmt.Fprintf(out, "peer identity: %s\n", ident.PeerID)
```

3단계: 기본 설정 생성

기본 설정 구조체 초기화

```
func InitWithIdentity(identity Identity) (*Config, error) {

bootstrapPeers, err := DefaultBootstrapPeers()

if err != nil {

return nil, err

}


datastore := DefaultDatastoreConfig()


conf := &Config{

API: API{

HTTPHeaders: map[string][]string{},

},
```

```
// 기본 네트워크 주소 설정
Addresses: addressesConfig(),


Datastore: datastore,
Bootstrap: BootstrapPeerStrings(bootstrapPeers),
Identity: identity,
Discovery: Discovery{
MDNS: MDNS{
Enabled: true,
},
},


// 마운트 포인트 설정
Mounts: Mounts{
IPFS: "/ipfs",
IPNS: "/ipns",
MFS: "/mfs",
},


Ipns: Ipns{
ResolveCacheSize: 128,
},
```

```
Gateway: Gateway{

RootRedirect: "",

NoFetch: false,

HTTPHeaders: map[string][]string{},

},

// 기타 기본 설정들...

}



return  conf, nil

}
```

기본 네트워크 주소 설정

```
func  addressesConfig() Addresses {

return Addresses{

Swarm: []string{

"/ip4/0.0.0.0/tcp/4001", // IPv4 TCP

"/ip6/::/tcp/4001", // IPv6 TCP

"/ip4/0.0.0.0/udp/4001/webrtc-direct", // WebRTC Direct

"/ip4/0.0.0.0/udp/4001/quic-v1", // QUIC v1

"/ip4/0.0.0.0/udp/4001/quic-v1/webtransport", // WebTransport

"/ip6/::/udp/4001/webrtc-direct",

"/ip6/::/udp/4001/quic-v1",

"/ip6/::/udp/4001/quic-v1/webtransport",
```

```
},
Announce: []string{},
AppendAnnounce: []string{},
NoAnnounce: []string{},
API: Strings{"/ip4/127.0.0.1/tcp/5001"}, // API 서버
Gateway: Strings{"/ip4/127.0.0.1/tcp/8080"}, // 게이트웨이
}
}
```

기본 데이터스토어 설정

```
func  DefaultDatastoreConfig() Datastore {
return Datastore{
StorageMax: "10GB", // 최대 저장 용량
StorageGCWatermark: 90, // GC 임계값 (90%)
GCPeriod: "1h", // GC 주기
BloomFilterSize: 0, // 블룸 필터 크기
Spec: flatfsSpec(), // FlatFS 사양
}
}
```

4단계: 프로파일 적용

프로파일 파싱 및 적용

```
func applyProfiles(conf *config.Config, profiles string) error {
if profiles == "" {
return nil
}

for _, profile := range strings.Split(profiles, ",") {
transformer, ok := config.Profiles[profile]
if !ok {
return fmt.Errorf("invalid configuration profile: %s", profile)
}

if err := transformer.Transform(conf); err != nil {
return err
}
}
return nil
}
```

주요 프로파일 예시

Server 프로파일:

```
"server": {
Description: `Disables local host discovery, recommended when
```

```
running IPFS on machines with public IPv4 addresses.`,
Transform: func(c *Config) error {
c.Addresses.NoAnnounce = appendSingle(c.Addresses.NoAnnounce, defaultServerFilte
c.Swarm.AddrFilters = appendSingle(c.Swarm.AddrFilters, defaultServerFilters)
c.Discovery.MDNS.Enabled = false
c.Swarm.DisableNatPortMap = true
return  nil
},
}
```

Test 프로파일:

```
"test": {
Description: `Reduces external interference of IPFS daemon, this
is useful when using the daemon in test environments.`,
Transform: func(c *Config) error {
c.Addresses.API = Strings{"/ip4/127.0.0.1/tcp/0"}
c.Addresses.Gateway = Strings{"/ip4/127.0.0.1/tcp/0"}
c.Addresses.Swarm = []string{"/ip4/127.0.0.1/tcp/0"}
c.Swarm.DisableNatPortMap = true
c.Bootstrap = []string{}
c.Discovery.MDNS.Enabled = false
return  nil
```

```
},

}
```

5단계: 저장소 초기화

FSRepo 초기화

```
func Init(repoPath string, conf *config.Config) error {
packageLock.Lock()
defer packageLock.Unlock()

if isInitializedUnsynced(repoPath) {
return nil
}

// 설정 파일 초기화
if err := initConfig(repoPath, conf); err != nil {
return err
}

// 데이터스토어 사양 파일 초기화
if err := initSpec(repoPath, conf.Datastore.Spec); err != nil {
return err
}
```

```
// 저장소 버전 파일 작성
if err := migrations.WriteRepoVersion(repoPath, RepoVersion); err != nil {
return err
}

return nil
}
```

설정 파일 작성

```
func initConfig(path string, conf *config.Config) error {
if configIsInitialized(path) {
return nil
}
configFilename, err := config.Filename(path, "")
if err != nil {
return err
}
// 원자적 파일 쓰기를 통한 설정 파일 생성
if err := serialize.WriteConfigFile(configFilename, conf); err != nil {
return err
}
```

```
return nil
}
```

데이터스토어 사양 파일 생성

```
func initSpec(path string, conf map[string]interface{}) error {
fn, err := config.Path(path, specFn)
if err != nil {
return err
}

if fsutil.FileExists(fn) {
return nil
}

dsc, err := AnyDatastoreConfig(conf)
if err != nil {
return err
}
bytes := dsc.DiskSpec().Bytes()

return os.WriteFile(fn, bytes, 0o600)
```

```
}
```

6단계: 기본 문서 추가 (선택적)

초기 문서 시딩

```
func addDefaultAssets(out io.Writer, repoRoot string) error {
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

r, err := fsrepo.Open(repoRoot)
if err != nil {
return err
}

nd, err := core.NewNode(ctx, &core.BuildCfg{Repo: r})
if err != nil {
return err
}
defer nd.Close()

// 초기 문서들을 IPFS에 추가
dkey, err := assets.SeedInitDocs(nd)
if err != nil {
```

```
return fmt.Errorf("init: seeding init docs failed: %s", err)
}
log.Debugf("init: seeded init docs %s", dkey)

if _, err = fmt.Fprintf(out, "to get started, enter:\n"); err != nil {
return err
}

_, err = fmt.Fprintf(out, "\n\tipfs cat /ipfs/%s/readme\n\n", dkey)
return err
}
```

포함되는 기본 문서들

```
var initDocPaths = []string{
gopath.Join("init-doc", "about"),
gopath.Join("init-doc", "readme"),
gopath.Join("init-doc", "help"),
gopath.Join("init-doc", "contact"),
gopath.Join("init-doc", "security-notes"),
gopath.Join("init-doc", "quick-start"),
gopath.Join("init-doc", "ping"),
}
```

7단계: IPNS 키스페이스 초기화

빈 디렉토리 생성 및 IPNS 발행

```
func initializeIpnsKeyspace(repoRoot string) error {
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

r, err := fsrepo.Open(repoRoot)
if err != nil {
return err
}

nd, err := core.NewNode(ctx, &core.BuildCfg{Repo: r})
if err != nil {
return err
}
defer nd.Close()

// 빈 디렉토리 노드 생성
emptyDir := unixfs.EmptyDirNode()

// 재귀적으로 핀 설정 (이미 핀된 경우를 대비)
err = nd.Pinning.Pin(ctx, emptyDir, true, "")
```

```
if err != nil {
return err
}

err = nd.Pinning.Flush(ctx)
if err != nil {
return err
}

// IPNS에 빈 디렉토리 발행
return nd.Namesys.Publish(ctx, nd.PrivateKey, path.FromCid(emptyDir.Cid()))
}
```

초기화 옵션별 동작

알고리즘 선택

Ed25519 (기본값)

- **장점**: 빠른 서명/검증, 작은 키 크기 (32바이트)
- **단점**: 상대적으로 새로운 알고리즘
- **사용 사례**: 대부분의 일반적인 용도

RSA

- **장점**: 널리 검증된 알고리즘, 높은 호환성

- **단점**: 큰 키 크기, 느린 연산
- **사용 사례**: 레거시 시스템과의 호환성이 필요한 경우

빈 저장소 옵션

기본 동작 (empty-repo=false)

- 도움말 문서들이 자동으로 추가됨
- 새 사용자를 위한 학습 자료 제공
- 약간의 추가 저장 공간 사용

빈 저장소 (empty-repo=true)

- 도움말 문서 추가 생략
- 최소한의 초기 설정만 수행
- 자동화된 배포에 적합

초기화 후 생성되는 파일 구조

```
~/.ipfs/
├── config # 주 설정 파일 (JSON)
├── datastore_spec # 데이터스토어 사양
├── version # 저장소 버전
├── keystore/ # 키 저장소 디렉토리
├── blocks/ # 블록 데이터 (FlatFS)
├── datastore/ # 메타데이터 (LevelDB)
```

```
└── repo.lock # 저장소 잠금 파일 (실행 시에만)
```

오류 처리 및 복구

일반적인 오류 상황

1. **저장소 이미 존재**: `errRepoExists` 반환
2. **권한 부족**: 쓰기 권한 검사 실패
3. **잘못된 프로파일**: 존재하지 않는 프로파일 지정
4. **키 생성 실패**: 암호화 라이브러리 오류

복구 전략

- **부분 초기화 실패**: 생성된 파일들을 정리하고 재시도
- **권한 문제**: 사용자에게 명확한 오류 메시지 제공
- **설정 오류**: 기본값으로 대체하여 계속 진행

보안 고려사항

개인키 보호

- 개인키는 Base64로 인코딩되어 저장
- 파일 권한을 0o600으로 설정하여 소유자만 접근 가능
- 향후 암호화 저장 계획 (TODO 항목)

저장소 잠금

- `repo.lock` 파일을 통한 동시 접근 방지

- 프로세스 종료 시 자동으로 잠금 해제
- 강제 종료 시 수동 잠금 파일 제거 필요

이 초기화 과정을 통해 IPFS 노드는 네트워크에 참여할 수 있는 모든 필수 구성 요소를 갖추게 되며, 사용자의 요구사항에 맞게 다양한 프로파일을 통해 최적화할 수 있습니다.